

Calcolatori Elettronici

2 luglio 2025

Teoria

Esercizio 1 (9 punti): Si realizzi una rete sequenziale che riceve in ingresso una sequenza infinita di caratteri appartenenti all'alfabeto $I = \{A, B, C\}$. Dopo aver ricevuto un gruppo di tre caratteri, la rete restituisce `sì` se uno dei tre caratteri è stato ricevuto più volte, viceversa restituirà `no`. Ad esempio, per una sequenza di tre caratteri pari a `AAB`, la macchina restituirà `sì`; per una sequenza pari a `ABC`, la macchina restituirà `no`. Fino a che non è stato ricevuto un gruppo di 3 caratteri, l'uscita assumerà il valore `?`. Si realizzi il circuito equivalente dell'equazione di uscita della macchina.

Esercizio 2 (7 punti): Codificare la parola binaria `1100` utilizzando un codice di Hamming con $h = 3$, utilizzando bit di parità pari, mostrando tutti i passaggi. Si supponga di ricevere il valore `0111101`: illustrare quali passaggi è necessario effettuare per determinare se c'è stato o meno un errore di trasmissione e ricostruire, se possibile, il valore originale.

Esercizio 3 (8 punti): Disegnare l'architettura interna del processore z64. Descrivere il microcodice atto a supportare l'esecuzione dell'istruzione `cmpq $1, %rax`, compresa la fase di fetch.

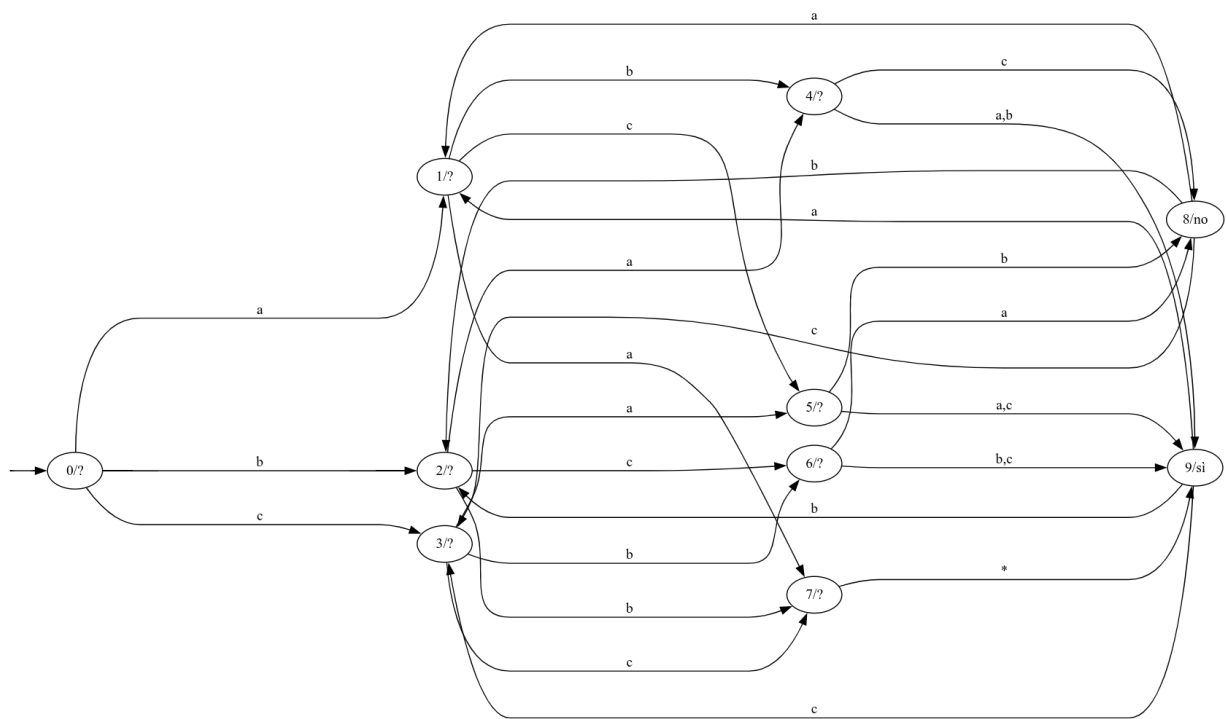
Esercizio 4 (6 punti): Discutere il funzionamento del comparatore veloce, mostrando come è possibile ricavare un modulo elementare composto unicamente da porte NAND.

Soluzioni

Esercizio 1

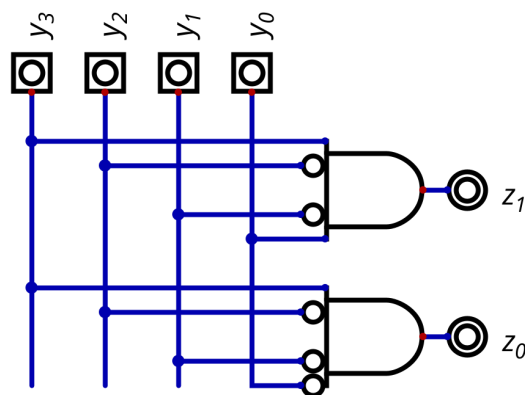
Per approcciare questo esercizio, è necessario prima di tutto ragionare. L'automa è una versione "estesa" di un contatore che, però, deve tenere traccia di quali caratteri sono stati osservati in precedenza. Per evitare un'esplosione di stati, è utile capire che, ad esempio, una sequenza `BAB` ed una sequenza `ABB` può attraversare lo stesso stato intermedio. Infatti, nel caso di entrambe le sequenze, quando si è osservato il secondo carattere (ossia `AB` o `BA`), ci si trova effettivamente nello stesso stato, corrispondente logicamente ad "ho osservato una `A` e una `B`", dal momento che, per contare, l'ordine non è importante. Proseguendo con questo ragionamento, si può arrivare alla conclusione che esiste un solo stato di accettazione ed un solo stato di rifiuto per tutta la macchina: l'ordine con cui si osservano i caratteri non è importante.

Il numero di stati, in ogni caso, non è minimale. Poiché però l'esercizio chiede di realizzare il circuito equivalente dell'uscita, dal momento che abbiamo capito che gli stati di accettazione e rifiuto sono unici, è utile utilizzare una macchina di Moore, arrivando a questo automa:



Si noti che gli stati "ho incontrato due A", "ho incontrato due B", "ho incontrato due C" sono un unico stato dal momento che è sufficiente ricordare l'informazione "ho incontrato due caratteri identici" (lo stato 7): qualsiasi sia il terzo ingresso, l'uscita dovrà comunque essere **si**.

Codifichiamo gli stati con il corrispondente valore binario. Codifichiamo poi l'uscita come segue: $0 = \{ \text{?}:00; \text{no}:10; \text{si}:01 \}$. Sfruttando questa codifica, è chiaro che possiamo studiare i valori dei bit di uscita associati a **si** e **no** in maniera indipendente. Poiché l'uscita **si** si ha soltanto nello stato 9, otteniamo immediatamente l'equazione $z_1 = y_3 \overline{y_2} \overline{y_1} y_0$. Poiché l'uscita **no** si ha soltanto nello stato 8, otteniamo immediatamente l'equazione $z_0 = y_3 \overline{y_2} y_1 \overline{y_0}$. Queste equazioni portano al circuito:



Esercizio 2

Sono necessari tre bit di parità. Infatti, $m + 1 \leq 2^p - p$ (con $m = 4$ poiché si vuole codificare una parola di 4 bit) è soddisfatta per $p = 3$. I bit di parità vanno nelle posizioni che sono potenze di due. Si può effettuare il calcolo utilizzando la codifica tabellare, dove il simbolo $\checkmark$ indica le posizioni coperte da ciascun bit di parità.

	p_1	p_2	1	p_4	1	0	0	
p_1	✓		✓		✓		✓	0
p_2		✓	✓			✓	✓	1
p_4				✓	✓	✓	✓	1

Si ottiene quindi la codifica `0111100`.

Supponiamo ora di ricevere il valore `0111101`. Determiniamo se c'è un errore di trasmissione utilizzando la decodifica del codice di Hamming:

- Verifica del bit di parità P_1 : controlla le posizioni 1, 3, 5, 7. I bit sono: 0, 1, 1, 1. La somma è dispari, quindi il bit di parità P_1 indica un errore.
- Verifica del bit di parità P_2 : controlla le posizioni 2, 3, 6, 7. I bit sono: 1, 1, 0, 1. La somma è dispari, quindi il bit di parità P_2 indica un errore.
- Verifica del bit di parità P_3 : controlla le posizioni 4, 5, 6, 7. I bit sono: 1, 1, 0, 1. La somma è dispari, quindi il bit di parità P_3 indica un errore.

La presenza di almeno un bit di parità errato indica che c'è stato almeno un errore di trasmissione. Si può provare a correggere un singolo errore di trasmissione, calcolando il numero di controllo $N_c = \langle P_1 P_2 P_3 \rangle = (111)_2 = 7$, che ci indica la necessità di invertire il bit in posizione 7, permettendoci di ottenere:

`0111101` $\rightarrow$ `0111100`

Andando a rimuovere i bit di parità in posizione 1, 2, 4, otteniamo `1100`, che era il valore inizialmente codificato.

Esercizio 3

Il microcodice atto a supportare `cmpq $1, %rax` è:

```

1  MAR ← RIP          BRIP = 1, WMAR = 1
2  MDR ← (MAR); RIP ← RIP + 8  INCRIP = 1, BAB = 1
3                               BAB = 1, MRD = 1
4                               BAB = 1, MRD = 1, BDBIN = 1, WMDR = 1
5  IR ← MDR           WIR = 1, BMDROUT = 1
6  MAR ← RIP          BRIP = 1, WMAR = 1
7  MDR ← (MAR); RIP ← RIP + 8  INCRIP = 1, BAB = 1
8                               BAB = 1, MRD = 1
9                               BAB = 1, MRD = 1, BDBIN = 1, WMDR = 1
10 TEMP1 ← MDR         BMDROUT = 1, WT1 = 1
11 TEMP2 ← RAX         SMUX = 2, REGMUX = 1, RM = 1, WT2 = 1
12 ALU OUT[SUB]        Aopcode = 0001

```

Progetto

Un processore `z64` governa una serra idroponica modulare destinata alla coltivazione sperimentale di lattughe a crescita rapida. Il processore è collegato alle seguenti periferiche. `CLIMA`, sincrona, fornisce temperatura e umidità relativa, ciascuna rappresentata come longword non segnata. Una griglia di `LAMPADE` a LED, costituita da 4 LED, viene controllata in maniera sincrona mediante un apposito registro di interfaccia di dimensione byte. In tale registro, il bit più significativo determina l'accensione/lo spegnimento di tutta la griglia. Allo stesso tempo,

ciascuno dei 4 bit meno significativi permette l'accensione controllata di ciascuno dei 4 LED. Una periferica **ACQUA**, anch'essa sincrona, consente l'apertura di una valvola che consente un deflusso di acqua all'interno della serra.

La serra è anche dotata di un **TIMER** programmabile, che consente di avvisare il processore quando è passata una certa quantità di tempo (in secondi). In maniera ciclica, lo **z64** interroga la periferica **CLIMA** per determinare la temperatura e l'umidità all'interno della serra. In funzione della temperatura, lo **z64** pilota **LAMPADE** nel modo seguente:

- se la temperatura è inferiore a 10 °C, tutti i LED vengono attivati;
- se la temperatura è compresa nell'intervallo [10 °C, 15°C], vengono attivati 3 LED;
- se la temperatura è compresa nell'intervallo [16 °C, 24 °C], vengono attivati 2 LED;
- se la temperatura è superiore a 24 °C, tutti i LED vengono spenti.

La misurazione dell'umidità, invece, determina il seguente funzionamento:

- se l'umidità relativa è inferiore al 30%, viene programmata **ACQUA** per erogare acqua per 10 secondi;
- se l'umidità relativa è inferiore al 60%, viene programmata **ACQUA** per erogare acqua per 5 secondi;
- altrimenti, **ACQUA** non viene attivata.

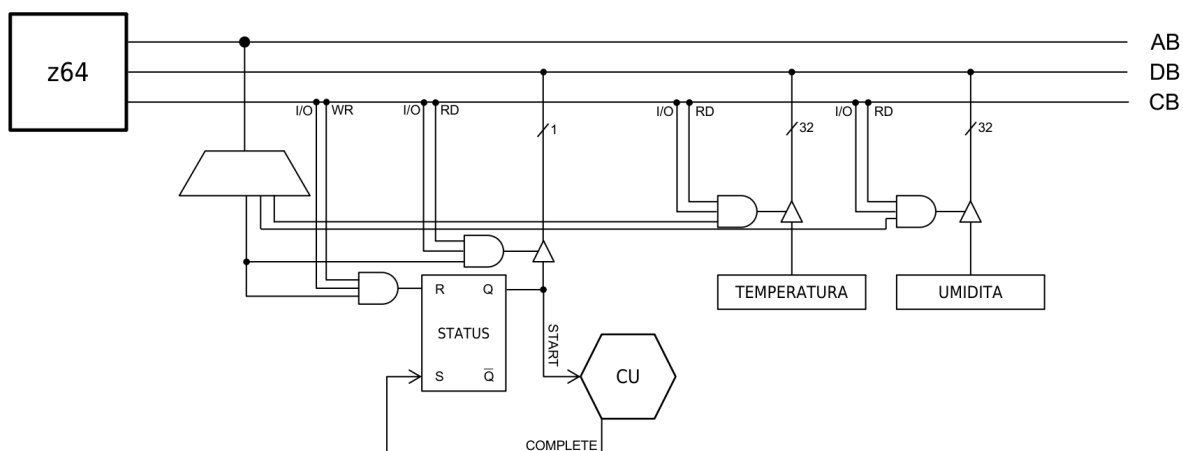
Tra una interrogazione di **CLIMA** e la successiva, devono trascorrere 60 secondi.

Realizzare:

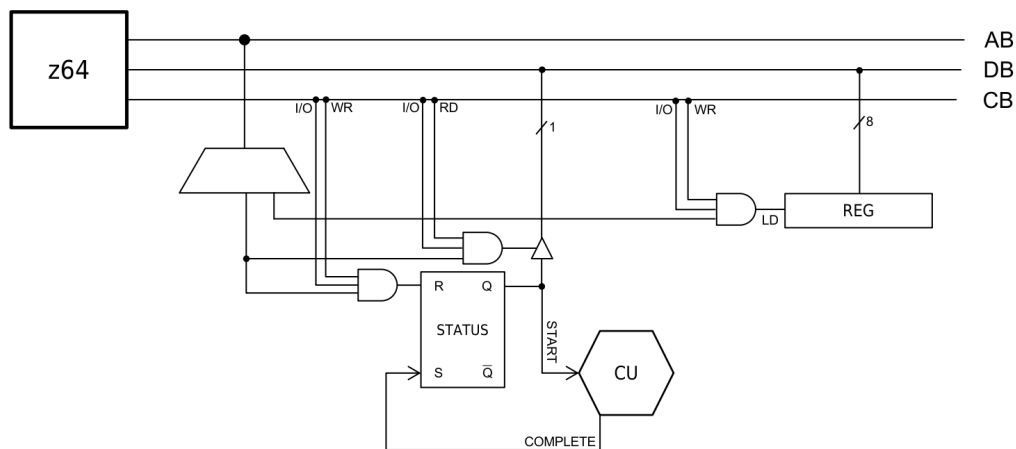
- Le interfacce dei dispositivi **CLIMA**, **LAMPADE** e **TIMER** (non è richiesta la progettazione di **ACQUA**);
- Tutto il codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

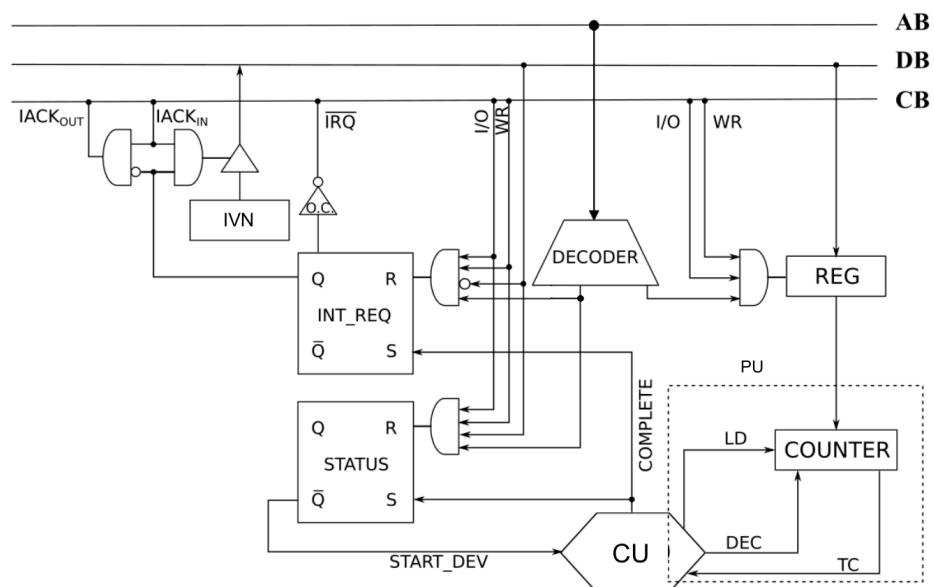
CLIMA è una periferica sincrona con due registri di input:



LAMPADE è una periferica sincrona di output con un singolo registro di interfaccia per trasmettere il codice:



Il **TIMER** è un dispositivo asincrono, con un registro di interfaccia di output per poter programmare la durata prima che venga generata la successiva richiesta di interruzione:



Per la scrittura del codice, si suppone che **ACQUA** si comporti come un interruttore on/off. Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .equ CLIMA_STATUS, 0x10
3  .equ CLIMA_TEMP, 0x11
4  .equ CLIMA_UMID, 0x12
5  .equ LAMPADE_REG, 0x21
6  .equ LAMPADE_STATUS, 0x21
7  .equ ACQUA_STATUS, 0x30
8  .equ TIMER_STATUS, 0x40
9  .equ TIMER_IRQ, 0x41
10 .equ TIMER_REG, 0x42
11 .data
12     trascorso: .byte 0
13 .text
14 main:
15     sti
16     .loop:
17         call acquisisci_temp_umid
18         # processamento temperatura

```

```

19     pushq %rax
20     cmpl $10, %eax
21     jc .temp_bassa
22     cmpl $15, %eax
23     jc .temp_10_15
24     jz .temp_10_15
25     cmpl $24, %eax
26     jc .temp_16_24
27     jz .temp_16_24
28     jmp .temp_alta
29
30 .temp_bassa:
31     movb $0x8F, %al # bit 7=1 + bit 0-3=1
32     jmp .led_done
33
34 .temp_10_15:
35     movb $0x87, %al # bit 7=1 + bit 0-2=1
36     jmp .led_done
37
38 .temp_16_24:
39     movb $0x83, %al # bit 7=1 + bit 0-1=1
40     jmp .led_done
41
42 .temp_alta:
43     movb $0x00, %al # bit7=0 → griglia spenta
44
45 .led_done:
46     outb %al, $LAMPADA_REG
47     outb %al, $LAMPADA_STATUS
48 .bw1:
49     inb $LAMPADA_STATUS, %al
50     btb $0, %al
51     jnc .bw1
52
53 # processamento umidità
54     popq %rax
55     shrq $32, %rax
56     cmpl $30, %eax
57     jc .umidita_bassa
58     cmpl $60, %eax
59     jc .umidita_media
60
61     movl $60, %edi
62     call attendi
63     jmp .loop
64
65 .umidita_bassa:
66     movl $10, %edi # eroga 10 s
67     movl $50, %esi # aspetta 50 s
68     jmp .acqua_prog
69
70 .umidita_media:
71     movl $5, %edi # eroga 5 s
72     movl $50, %esi # aspetta 55 s
73
74 .acqua_prog:
75     movb $1, %al
76     outb %al, $ACQUA_STATUS
77     call attendi
78     movb $0, %al
79     outb %al, $ACQUA_STATUS
80     movl %esi, %edi
81     call attendi
82     jmp .loop
83
84 attendi:
85     movb $0, %al # trascorso

```

```

86     movl %esi, %eax
87     outl %eax, $TIMER_REG
88     outb %al, $TIMER_STATUS
89 .loop2:
90     cmpb $1, trascorso
91     jnz .loop2
92     ret
93
94 # Restituisce nella parte alta di rax l'umidità, in quella bassa la temperatura
95 acquisisci_temp_umid:
96     outb %al, $CLIMA_STATUS
97     .bw2:
98     inb $CLIMA_STATUS, %al
99     btb $0, %al
100    jnc .bw2
101    xorq %rax, %rax
102    inl $CLIMA_UMID, %eax
103    movl %eax, %ebx
104    shlq $32, %rbx
105    inl $CLIMA_TEMP, %eax
106    orq %rbx, %rax
107    ret
108
109 .driver 0 # timer
110     push %rax
111     movb $1, trascorso
112     outb %al, $TIMER_IRQ
113     pop %rax
114     iret

```

